



Documentación de la Prueba ATM

Generación automática de datos de prueba y automatización de búsqueda
de vuelos LATAM

Guía técnica · Conceptos para aprender · Manual de ejecución y pruebas

Componente	Tecnología	Qué resuelve
PARTE 1 — Generador de datos	Java 21 · Maven · SQLite	Genera datos ficticios con reglas de negocio
PARTE 2 — Automatización UI	Serenity BDD · Cucumber	Automatiza 3 casos de búsqueda de vuelos

Contenido

1. Introducción y objetivo
2. Cómo está organizado el proyecto
3. PARTE 1 — Generador de datos de prueba
 - 3.1 Qué pide el enunciado y cómo se cumple
 - 3.2 Conceptos clave: POO, patrones y SOLID
 - 3.3 Recorrido del código (cómo funciona)
 - 3.4 Cómo lo harías tú desde cero
4. PARTE 2 — Automatización Serenity + Cucumber
 - 4.1 Diseño de los 3 casos de prueba
 - 4.2 Conceptos clave: BDD, Gherkin y Screenplay
 - 4.3 Recorrido del framework
5. Manual de ejecución y pruebas
 - 5.1 Requisitos previos (instalación)
 - 5.2 Ejecutar el generador de datos
 - 5.3 Ejecutar la automatización
 - 5.4 Publicar en un repositorio público
6. Solución de problemas frecuentes
7. Glosario
8. Checklist de entrega

1. Introducción y objetivo

Este documento explica, de principio a fin, la solución construida para la **Prueba ATM** de GreenSQA. Está pensado con dos propósitos: que puedas **entender y sustentar** cada decisión en la conversación técnica, y que aprendas los **conceptos** necesarios para construir algo así por tu cuenta.

La prueba se compone de dos entregables que se conectan entre sí:

- **PARTE 1:** un programa en Java que genera datos de prueba ficticios (personas y empresas) cumpliendo un conjunto de reglas de negocio, los guarda en una base de datos y los exporta a un archivo CSV.
- **PARTE 2:** un framework de automatización de pruebas de interfaz (UI) sobre el sitio de LATAM, que automatiza la búsqueda de vuelos y usa como datos de entrada el CSV producido en la PARTE 1.

Contexto de la entrega

Según la comunicación recibida, solo se desarrolla la **Prueba ATM**; la prueba de performance (NF) no es necesaria. Fecha límite de entrega: **domingo 21 de junio**. Esta documentación cubre únicamente la Prueba ATM.

A lo largo del documento encontrarás tres tipos de contenido: **explicación de qué hace el código**, los **conceptos teóricos** detrás (para que aprendas), y un **manual práctico** con los comandos exactos para ejecutar y probar todo.

2. Cómo está organizado el proyecto

El proyecto es un repositorio con dos módulos independientes (cada uno con su propio archivo de construcción Maven) y una carpeta de documentación. Esta separación es intencional: cada módulo se construye, prueba y ejecuta por separado, pero comparten el CSV de datos como punto de conexión.

```
latam-qa-poc/
|-- data-generator/          PARTE 1 (Java + Maven + SQLite)
|   |-- src/main/java/...    codigo fuente del generador
|   |-- src/test/java/...    pruebas unitarias
|   |-- output/              CSV generados (datos de prueba)
|   |-- pom.xml              configuracion de dependencias
|   `-- README.md
|
|-- automation/              PARTE 2 (Serenity + Cucumber)
|   |-- src/test/java/...    tasks, questions, page objects, steps
|   |-- src/test/resources/
|   |   |-- features/        escenarios en Gherkin (.feature)
|   |   `-- serenity.conf    configuracion del navegador
|   `-- pom.xml
|
|-- docs/
|   |-- casos-de-prueba.md    disenio de los 3 casos
|   `-- Documentacion-Prueba-ATM.pdf (este documento)
`-- README.md                guia general del repositorio
```

La idea de fondo: **separación de responsabilidades**. El generador no sabe nada de navegadores ni de LATAM; la automatización no sabe cómo se generan los datos, solo los consume desde un CSV. Esto hace que cada parte sea más fácil de entender, mantener y reutilizar.

3. PARTE 1 — Generador de datos de prueba

El generador produce registros de personas y empresas ficticias. Cada registro debe respetar reglas estrictas. Antes de ver el código, conviene tener clarísimas esas reglas, porque son el corazón del ejercicio y lo primero que te van a preguntar.

3.1 Qué pide el enunciado y cómo se cumple

Regla del enunciado	Cómo se implementó
Nombre y Apellido	Catálogo de nombres/apellidos; las empresas usan razón social.
Apellido en blanco si es empresa	El builder fuerza apellido vacío para empresas.
Nombre + apellido no se repite	Registro Singleton thread-safe que reserva cada combinación.
Edad > 10 y < 80	Se genera en el rango [11, 79].
Documento de empresa inicia por 9	Estrategia de documento específica para empresa.
Documento de menor desde 11000000	Estrategia de menor: 11.000.000 + offset aleatorio.
Documento de mayor: dígitos > 8 y < 12	Estrategia de adulto: genera 9, 10 u 11 dígitos.
Documentos no se repiten	El mismo Singleton reserva cada documento.
Ciudad y País de residencia	Catálogo país -> ciudades.
País ≠ Colombia ⇒ idioma ≠ Español	Catálogo país -> idiomas; solo Colombia tiene Español.

Requisitos técnicos adicionales y bonus

Requisito	Estado	Dónde
4 pilares de POO	Cumplido	Jerarquía Identity / Person / Company
Patrones de diseño distintos	Cumplido (4)	Singleton, Strategy, Factory, Builder
Almacenar en base de datos	Cumplido	SQLite (SQLiteIdentityRepository)
Bonus: gestionar ejecuciones pasadas	Cumplido	listar/contar/eliminar por run_id
Al menos 2 principios SOLID	Bonus: los 5	Ver sección 3.2
Indicar cuántos registros generar	Cumplido	Parámetro --cantidad
Archivo de texto separado por comas	Cumplido	CsvExporter
Bonus: enviar el archivo por correo	Cumplido	EmailSender (SMTP)
Bonus: ejecución en paralelo	Cumplido	generarParalelo() con hilos

3.2 Conceptos clave: POO, patrones y SOLID

Esta es la parte teórica que te conviene dominar. La explico con palabras sencillas y luego muestro dónde aparece cada concepto en el proyecto.

Los 4 pilares de la Programación Orientada a Objetos (POO)

Pilar	Idea en una frase	En el proyecto
Abstracción	Definir QUÉ hace algo, sin el cómo.	La clase abstracta Identity define que toda identidad tiene datos y un método <code>esValido()</code> , sin decir cómo.
Encapsulamiento	Ocultar el estado interno y exponerlo de forma controlada.	Los atributos de <code>Identity</code> son privados; se acceden por getters/setters.
Herencia	Una clase reutiliza y extiende a otra.	PersonIdentity y CompanyIdentity heredan de <code>Identity</code> .
Polimorfismo	El mismo mensaje produce comportamientos distintos.	<code>esValido()</code> valida distinto para persona o empresa, aunque se llame igual.

Patrones de diseño (soluciones probadas a problemas comunes)

Patrón	Para qué sirve	Por qué aquí
Singleton	Garantizar una única instancia global compartida.	Un solo registro de unicidad para nombres y documentos, incluso entre hilos.
Strategy	Intercambiar algoritmos sin condicionales rígidos.	Cada tipo (empresa/menor/adulto) genera el documento con su propia estrategia.
Factory Method	Decidir qué objeto crear según el contexto.	Una fábrica elige la estrategia de documento adecuada.
Builder	Construir objetos complejos paso a paso.	Se arma la identidad atributo por atributo y se decide persona vs empresa.

Principios SOLID (cinco reglas para diseñar código mantenible)

El enunciado pedía al menos 2; el proyecto aplica los 5 (bonus completo). En palabras simples:

Letra	Principio	Significado simple	En el proyecto
S	Responsabilidad única	Cada clase hace una sola cosa.	Generar, exportar, persistir y enviar correo son clases separadas.
O	Abierto/Cerrado	Extender sin modificar lo existente.	Una nueva regla de documento = nueva estrategia, sin tocar el generador.
L	Sustitución de Liskov	Una subclase puede reemplazar a su base.	Cualquier repositorio cumple el contrato sin romper a quien lo usa.
I	Segregación de interfaces	Interfaces pequeñas y específicas.	El repositorio expone solo operaciones de datos cohesivas.
D	Inversión de dependencias	Depender de abstracciones, no de detalles.	El servicio depende de la interfaz <code>IdentityRepository</code> , no de <code>SQLite</code> .

Cómo explicarlo en la entrevista

Si te preguntan «¿dónde está el polimorfismo?», responde con un ejemplo concreto: «cuando llamo `esValido()` sobre una identidad, si es empresa verifica que el documento empiece por 9, y si es persona verifica el rango de edad y los dígitos del documento; es el mismo método con comportamiento distinto».

3.3 Recorrido del código (cómo funciona)

Veamos el flujo completo, desde que pides N registros hasta que obtienes el CSV. El orden de lectura recomendado del código es: modelo → patrones → generador → servicio → persistencia → exportador → Main.

Paso 1 — El modelo: la clase abstracta Identity

Define el estado común (nombre, apellido, edad, documento, etc.) con atributos privados (encapsulamiento) y declara métodos abstractos que cada subtipo implementa (abstracción + polimorfismo).

```
public abstract class Identity {
    private String nombre;
    private String apellido;    // vacio para empresas
    private int edad;
    private String documento;
    // ... getters y setters (encapsulamiento) ...

    public abstract String getTipoIdentidad(); // "PERSONA" o "EMPRESA"
    public abstract boolean esValido();        // reglas propias de cada tipo
}
```

Paso 2 — Las subclases: reglas distintas por tipo

PersonIdentity valida edad y dígitos del documento; CompanyIdentity valida apellido en blanco y documento que inicia por 9. Mismo método esValido(), distinto comportamiento.

```
public class CompanyIdentity extends Identity {
    public String getTipoIdentidad() { return "EMPRESA"; }
    public boolean esValido() {
        boolean apellidoVacio = getApellido() == null || getApellido().isBlank();
        return apellidoVacio && getDocumento().startsWith("9");
    }
}
```

Paso 3 — Strategy + Factory: el documento correcto

Cada regla de documento es una estrategia. La fábrica decide cuál usar según tipo y edad. Así, el generador no tiene una cadena de if/else gigante: pide «la estrategia para este caso» y la usa.

```
public DocumentStrategy crear(boolean esEmpresa, int edad) {
    if (esEmpresa) return company;    // inicia por 9
    return edad < 18 ? minor : adult; // menor: >=11000000 | adulto: 9-11 digitos
}
```

Paso 4 — Builder + Singleton: armar y garantizar unicidad

El builder ensambla la identidad. El registro Singleton reserva la combinación nombre+apellido y el documento; si ya existían, se descarta y se reintenta. Como el Singleton usa estructuras concurrentes, funciona aunque varios hilos generen a la vez.

```
if (!registry.reservarNombreCompleto(candidato.claveNombreCompleto())) continue;
if (!registry.reservarDocumento(candidato.getDocumento())) continue;
return candidato; // unico y valido
```

Paso 5 — Persistencia (SQLite) y gestión de ejecuciones

El servicio guarda los registros a través de la interfaz IdentityRepository (inversión de dependencias). La implementación SQLite marca cada lote con un `run_id`, lo que permite listar, contar o borrar ejecuciones pasadas (el bonus de gestión).

Paso 6 — Exportar a CSV y (opcional) enviar por correo

CsvExporter escribe el archivo separado por comas en UTF-8. Si se pide `--email` y hay credenciales SMTP configuradas, EmailSender adjunta el CSV y lo envía.

Ejecución en paralelo (bonus)

El servicio puede repartir la generación entre varios hilos. Cada hilo genera su parte; la unicidad global se mantiene gracias al Singleton. En las pruebas, generar en serie y en paralelo produjo cero duplicados.

3.4 Cómo lo harías tú desde cero

Si tuvieras que reconstruir este generador, este es el camino que te recomiendo seguir, en orden:

- 1 Escribe las reglas en papel.** Antes de programar, lista cada regla del enunciado como una condición verificable (edad entre 11 y 79, documento de empresa empieza por 9, etc.).
- 2 Modela el dominio.** Crea la clase base abstracta Identity y dos subclases. Pon el método `esValido()` en cada una traduciendo las reglas a código.
- 3 Aísla lo que cambia.** La generación del documento cambia por tipo: conviértelo en estrategias (Strategy) y añade una fábrica (Factory) que elija.
- 4 Garantiza unicidad.** Usa una estructura de conjunto (Set) para nombres y otra para documentos. Si vas a paralelizar, que sean concurrentes y vivan en un Singleton.
- 5 Persiste.** Define una interfaz de repositorio y una implementación con SQLite (o la base que prefieras). Depende de la interfaz, no de la base.
- 6 Exporta.** Una clase que recorra los registros y escriba el CSV.
- 7 Añade la CLI.** Un main que lea `--cantidad` y opciones, orqueste todo e imprima el resultado.
- 8 Prueba.** Escribe pruebas que generen muchos registros y verifiquen cada regla; así demuestras que funciona.

Consejo de aprendizaje

No intentes escribir todo de una vez. Haz que primero funcione lo mínimo (generar un nombre y una edad válida), pruébalo, y ve agregando reglas y patrones de a uno. Programar es iterar.

4. PARTE 2 — Automatización Serenity + Cucumber

La segunda parte automatiza la búsqueda de vuelos en el portal de LATAM. El enunciado pide diseñar 3 casos de prueba, construir el framework e implementarlos con **Serenity** y **Cucumber**, usando como datos de entrada el CSV generado en la PARTE 1.

4.1 Diseño de los 3 casos de prueba

ID	Caso	Qué valida
CP01	Búsqueda ida y vuelta, 1 adulto	Camino feliz: que se muestren resultados de vuelos.
CP02	Búsqueda solo ida con dato del CSV	Integración con PARTE 1: usa un pasajero del CSV.
CP03	Origen igual a destino	Validación negativa: el sistema impide la búsqueda.

Estos tres casos cubren un escenario positivo, uno orientado a datos (data-driven) y uno negativo, que es una buena cobertura mínima para una funcionalidad de búsqueda.

4.2 Conceptos clave: BDD, Gherkin y Screenplay

BDD (Behavior-Driven Development)

Es una forma de escribir pruebas describiendo el **comportamiento** esperado en lenguaje cercano al negocio, no en términos técnicos. La idea es que cualquier persona del equipo (incluso no técnica) pueda leer qué se está probando.

Gherkin: el idioma de los escenarios

Gherkin es el formato de los archivos .feature. Usa palabras clave Dado / Cuando / Entonces (Given / When / Then) para describir el contexto, la acción y el resultado esperado. Ejemplo real del proyecto:

```
# language: es
Escenario: CP01 - Búsqueda de vuelo ida y vuelta para un pasajero adulto
  Cuando selecciona el tipo de viaje "Ida y vuelta"
  Y completa el origen "Bogota" y el destino "Santiago"
  Y selecciona una fecha de ida a 20 días y una fecha de regreso a 27 días
  Y configura 1 pasajero adulto
  Y ejecuta la búsqueda de vuelos
  Entonces se muestran resultados de vuelos disponibles para la ruta seleccionada
```

Serenity y el patrón Screenplay

Serenity es la herramienta que ejecuta las pruebas, controla el navegador (vía Selenium) y genera reportes visuales. El patrón **Screenplay** organiza el código en torno a un «actor» (el usuario) que ejecuta «tarefas» (Tasks) y hace «preguntas» (Questions) sobre lo que ve. Es más legible y mantenible que el clásico Page Object por sí solo.

Pieza Screenplay	Qué representa	Ejemplo del proyecto
Actor	Quien usa la aplicación.	«Pasajero»
Task (Tarea)	Una acción de negocio.	IngresarRuta, BuscarVuelos
Question (Pregunta)	Una verificación sobre la pantalla.	ResultadosDeVuelos.sonVisibles()

Pieza Screenplay	Qué representa	Ejemplo del proyecto
Target (Localizador)	Dónde está un elemento en la página.	HomePage.BOTON_BUSCAR

4.3 Recorrido del framework

El flujo de una prueba va de arriba hacia abajo por estas capas:

```

.feature (Gherkin)          <- describe el escenario en lenguaje natural
  v
Step Definitions            <- traduce cada paso a tareas del actor
  v
Tasks / Questions          <- acciones de negocio y verificaciones
  v
Page Objects (Targets)     <- localizadores de los elementos web
  v
Serenity + Selenium WebDriver <- controla el navegador real

```

La clase TestDataProvider lee el CSV de la PARTE 1 y entrega un «pasajero» al caso CP02. Así, el dato que generó el primer módulo se convierte en entrada del segundo, cumpliendo lo que pide el enunciado.

Nota importante sobre los selectores

El sitio de LATAM es una aplicación web dinámica. Los localizadores (Targets) están centralizados en la clase HomePage y son representativos; antes de la ejecución real conviene verificarlos contra el sitio en vivo. Si el sitio cambia, solo se ajusta ese archivo, no las pruebas.

5. Manual de ejecución y pruebas

Esta sección es la guía práctica paso a paso. Sigue el orden: primero instalas las herramientas, luego ejecutas el generador, después la automatización.

5.1 Requisitos previos (instalación)

Herramienta	Versión	Para qué
JDK (Java)	21	Compilar y ejecutar ambos módulos.
Maven	3.8 o superior	Descargar dependencias y construir.
Google Chrome	Actual	Navegador para la automatización.
Git	Cualquiera	Publicar en el repositorio.

Verifica que todo esté instalado con estos comandos (deberías ver las versiones):

```
java -version      # debe mostrar 21.x
mvn -version       # debe mostrar 3.8 o superior
git --version
```

Si no tienes Java o Maven

En Windows, lo más sencillo es instalar un JDK 21 (por ejemplo Temurin/Adoptium) y Maven desde su sitio oficial, agregando ambos al PATH. En macOS puedes usar Homebrew: «brew install openjdk@21 maven». En Linux: «sudo apt install openjdk-21-jdk maven».

5.2 Ejecutar el generador de datos (PARTE 1)

Ubícate en la carpeta del módulo y construye el proyecto. La primera vez, Maven descargará las dependencias (necesita internet).

```
cd data-generator
mvn clean test      # ejecuta las pruebas unitarias (valida las reglas)
mvn clean package   # genera el ejecutable target/data-generator.jar
```

Ahora genera datos. Algunos ejemplos:

```
# 100 registros (modo serial), CSV en output/ y BD en data/
java -jar target/data-generator.jar --cantidad 100

# 1000 registros en paralelo con 4 hilos
java -jar target/data-generator.jar --cantidad 1000 --paralelo --hilos 4

# Indicar archivo de salida y base de datos
java -jar target/data-generator.jar --cantidad 200 \
  --salida output/datos_actual.csv --db data/identidades.db

# Gestion de datos almacenados (bonus)
java -jar target/data-generator.jar --listar      # cuantos hay en la BD
java -jar target/data-generator.jar --limpiar     # borra todo
```

Enviar el CSV por correo (bonus)

Configura las credenciales SMTP como variables de entorno y agrega --email. Nunca pongas contraseñas en el código. Para Gmail necesitas una «contraseña de aplicación».

```
export SMTP_HOST=smtp.gmail.com
export SMTP_PORT=587
export SMTP_USER=tu_correo@gmail.com
export SMTP_PASSWORD=tu_password_de_aplicacion
export MAIL_TO=destinatario@dominio.com
java -jar target/data-generator.jar --cantidad 50 --email
```

En Windows (PowerShell) usa: `$env:SMTP_HOST="smtp.gmail.com"` (y así con cada variable).

Cómo verificar que funcionó

- Revisa que se creó el archivo CSV en la carpeta output/ y ábrelo: debe tener encabezado y N filas.
- Comprueba que no hay documentos ni combinaciones nombre+apellido repetidas.
- Ejecuta «--listar» para confirmar que los registros quedaron en la base de datos.
- Las pruebas unitarias («mvn test») deben pasar todas: son la garantía automática de las reglas.

5.3 Ejecutar la automatización (PARTE 2)

Primero asegúrate de tener un CSV de datos disponible (generado en el paso anterior). Luego:

```
cd ../automation

# Ejecuta los 3 casos. Por defecto lee
# ../data-generator/output/datos_actual.csv
mvn clean verify

# Indicar otro CSV de datos
mvn clean verify -Dtestdata.csv=/ruta/al/datos.csv

# Ver el navegador (no headless) mientras corre
mvn clean verify -Dheadless.mode=false
```

Al terminar, abre el reporte visual de Serenity en tu navegador:

```
automation/target/site/serenity/index.html
```

Ese reporte muestra cada escenario, los pasos ejecutados, capturas de pantalla de los fallos y el resultado general. Es lo que adjuntarías como evidencia.

5.4 Publicar en un repositorio público

El enunciado pide publicar ambos desarrollos en un repositorio público con instrucciones de ejecución (que ya están en los archivos README). Pasos:

```
cd latam-qa-poc
git init
git add .
git commit -m "Prueba ATM GreenSQA: generador de datos + automatizacion"
# crea un repositorio vacio en GitHub y luego:
git remote add origin https://github.com/TU_USUARIO/latam-qa-poc.git
git branch -M main
git push -u origin main
```

El archivo .gitignore ya excluye carpetas de construcción (target/), bases de datos y CSV generados, para que el repositorio quede limpio.

6. Solución de problemas frecuentes

Síntoma	Causa probable	Solución
«mvn: command not found»	Maven no está instalado o no está en el PATH.	Instala Maven y agrégalo al PATH; verifica con «mvn -version».
Error 403 / no descarga dependencias	Sin acceso a Maven Central o proxy corporativo.	Conéctate a una red abierta o configura el proxy en ~/.m2/settings.xml.
«javac: command not found»	Tienes solo el JRE, no el JDK.	Instala el JDK 21 completo (no solo el runtime).
La automatización no encuentra el CSV	Ruta del CSV incorrecta.	Genera primero los datos o pasa -Dtestdata.csv con la ruta correcta.
No abre Chrome / driver	Chrome no instalado o versión incompatible.	Instala Chrome actualizado; Serenity descarga el driver solo.
Un caso de UI falla por selector	El sitio de LATAM cambió el DOM.	Ajusta el localizador en la clase HomePage.
«No fue posible generar identidad única»	Pediste más registros de los que el catálogo permite.	Reduce --cantidad o amplía los catálogos de nombres.

7. Glosario

API / CLI. CLI es la interfaz de línea de comandos: el programa se ejecuta escribiendo comandos en una terminal con parámetros.

Build / Maven. Construir el proyecto: compilar el código y empaquetarlo. Maven automatiza esto y descarga las dependencias.

BDD. Behavior-Driven Development: escribir pruebas describiendo el comportamiento esperado en lenguaje de negocio.

Cucumber. Herramienta que lee los archivos .feature en Gherkin y los conecta con el código de prueba.

CSV. Archivo de texto con valores separados por comas; cada línea es un registro.

Gherkin. Lenguaje de los escenarios BDD con las palabras Dado / Cuando / Entonces.

Headless. Ejecutar el navegador sin ventana visible; útil en servidores y más rápido.

JDK / JRE. JDK incluye el compilador (javac) para desarrollar; el JRE solo ejecuta. Para este proyecto necesitas el JDK.

POO. Programación Orientada a Objetos: organizar el código en clases y objetos.

Screenplay. Patrón de Serenity que modela un actor que ejecuta tareas y hace preguntas.

Selenium / WebDriver. Tecnología que controla un navegador real de forma programática.

Serenity BDD. Framework que ejecuta las pruebas, maneja el navegador y genera reportes visuales.

SQLite. Base de datos ligera que se guarda en un solo archivo, sin necesidad de un servidor.

Thread / Paralelo. Un hilo es una línea de ejecución; ejecutar en paralelo usa varios hilos a la vez para ir más rápido.

8. Checklist de entrega

Antes de enviar la prueba (fecha límite: domingo 21 de junio), confirma cada punto:

#	Ítem a verificar	Listo
1	El generador compila y las pruebas unitarias pasan (mvn test).	[]
2	Se genera el CSV con la cantidad indicada y sin duplicados.	[]
3	Los datos quedan almacenados en la base de datos SQLite.	[]
4	Se evidencian los 4 pilares POO y los 4 patrones de diseño.	[]
5	Se aplican los principios SOLID (mínimo 2; ideal los 5).	[]
6	Funciona el parámetro de cantidad y la exportación a CSV.	[]
7	Bonus: envío por correo y ejecución en paralelo probados.	[]
8	Están diseñados y documentados los 3 casos de prueba.	[]
9	El framework Serenity + Cucumber ejecuta los 3 escenarios.	[]
10	La automatización consume el CSV de la PARTE 1 como entrada.	[]
11	Se genera el reporte de Serenity como evidencia.	[]
12	Todo está en un repositorio público con README de ejecución.	[]

Recomendación final

Antes de la conversación técnica, repasa la sección 3.2 (conceptos) y prepárate para señalar en el código dónde está cada pilar, patrón y principio. Saber «dónde» y «por qué» es lo que demuestra dominio.